

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO BÀI TẬP LỚN

HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN

Thực hiện bởi: NHÓM 4

- Nguyễn Thành Lộc
- Nguyễn Phúc Bình
- Nguyễn Hải Anh
- Trần Minh Trung

MSV: 23020000
MSV: 23020000
MSV: 25020020
MSV: 23020000

Vai trò: Trưởng nhóm
Vai trò: Thành viên
Vai trò: Thành viên
Vai trò: Thành viên

1. GIỚI THIỆU MỤC TIÊU VÀ PHẠM VI THỰC HIỆN

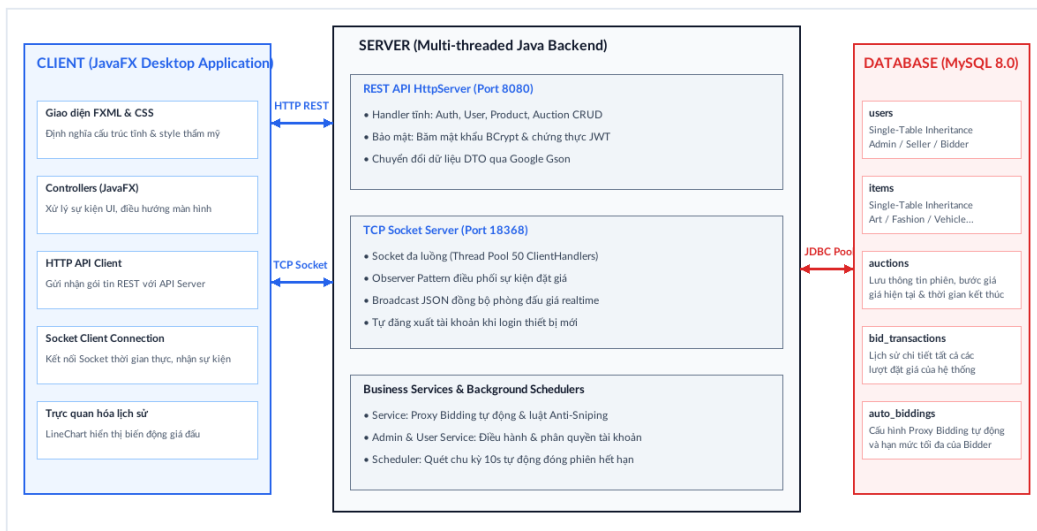
Mục tiêu của đề án là xây dựng hệ sinh thái đấu giá trực tuyến chuyên nghiệp dạng Desktop Application, tối ưu hóa tối đa trải nghiệm người dùng cuối trên nền tảng JavaFX và đảm bảo tính đồng bộ dữ liệu giao dịch ở mức cao thông qua kết hợp REST API và mạng kết nối TCP Socket thời gian thực (realtime). Hệ thống cam kết mang lại sự minh bạch tuyệt đối, tốc độ xử lý đặt giá tức thì và khả năng vận hành ổn định cao dưới khối lượng người dùng truy cập đồng thời lớn.

Phạm vi thực hiện của hệ thống bao gồm 4 phân hệ chính hoạt động thống nhất:

- **Phân hệ người đặt giá (Bidder):** Xem danh sách đấu giá, lọc trạng thái (đang chờ duyệt, đang diễn ra, đã kết thúc), vào phòng đấu giá trực tiếp, đặt giá realtime mượt mà, quản lý số dư ví điện tử cá nhân và nạp tiền, cấu đặt đấu giá tự động thông minh (Auto-Bidding) và giám sát các sản phẩm thắng cuộc.
- **Phân hệ người bán hàng (Seller):** Đăng ký mặt hàng mới với các thuộc tính đặc trưng chuyên sâu riêng biệt (như tác giả cho Art, số ODO cho Vehicle...), quản lý toàn diện vòng đời sản phẩm, giám sát trạng thái và kết quả đấu giá của sản phẩm sở hữu. Người bán tuyệt đối không được phép tự đặt giá trên sản phẩm của mình.
- **Phân hệ quản trị viên (Admin):** Quản lý dữ liệu tài khoản người dùng tập trung, tìm kiếm và cấp/gỡ quyền Admin an toàn qua mã quản trị, phê duyệt hoặc từ chối các phiên đấu giá do người bán gửi lên hệ thống, theo dõi tổng quan dashboard hoạt động.
- **Phân hệ máy chủ xử lý (Server):** Lưu trữ và điều hành kết nối database qua MySQL kết hợp Connection Pool HikariCP, cung cấp cổng REST API cho hoạt động quản lý/nghiệp vụ tĩnh và máy chủ TCP Socket Server đa luồng điều phối sự kiện đấu giá realtime.

2. KIẾN TRÚC TỔNG THỂ HỆ THỐNG

Hệ thống AuctionSpace được xây dựng chặt chẽ theo mô hình Client-Server phân tầng, tách biệt hoàn toàn giao diện người dùng tĩnh và luồng xử lý nghiệp vụ thời gian thực. Sơ đồ kiến trúc tổng thể được đặc tả như sau:



Hình 2.1: Sơ đồ kiến trúc tổng thể hệ thống Client-Server của AuctionSpace

Mô tả hoạt động cốt lõi của kiến trúc:

- **Đồng bộ realtime:** Khi người dùng đặt giá qua TCP Socket, Socket Server nhận command 'BID' qua luồng ClientHandler tương ứng. Yêu cầu chuyển tới BiddingService xử lý. Nếu thành công, Server cập nhật giá mới và sử dụng mô hình Observer phát sóng (broadcast) thông điệp JSON tới toàn bộ các client đang kết nối để cập nhật tức thì giao diện phòng đấu giá.
- **Xác thực bảo mật:** Giao diện JavaFX sử dụng các yêu cầu HTTP REST để đăng ký, đăng nhập và lấy thông tin tĩnh. Mật khẩu được băm bằng thuật toán BCrypt. Các API nghiệp vụ sau khi đăng nhập được kiểm soát quyền qua Token JWT đính kèm ở HTTP Header, đảm bảo tính an toàn dữ liệu tuyệt đối giữa Client và Server.
- **Thiết kế DB tối ưu:** Hệ thống áp dụng kỹ thuật Single-Table Inheritance cho 2 cây kế thừa chính: 'users' (chứa chung trường của Admin, Seller, Bidder) và 'items' (chứa chung thuộc tính của Art, Vehicle, Electronics...) giúp giảm thiểu phép kết nối bảng JOIN phức tạp và tăng hiệu năng truy vấn database.

3. TRÌNH BÀY CHI TIẾT CHỨC NĂNG

Dưới đây là phần đặc tả chi tiết 16 hạng mục chức năng (bao gồm 13 hạng mục bắt buộc và 3 hạng mục tự chọn cộng điểm) đã hoàn thành trong hệ thống AuctionSpace, kèm giải pháp kỹ thuật và lý do lựa chọn thiết kế:

3.1 Thiết kế lớp và cây kế thừa (0.5 điểm) - Bắt buộc

Hiện trạng: Khai báo lớp abstract User (lớp con: Bidder, Seller, Admin) và Item (lớp con: Art, Collectible, Electronics, Fashion, Vehicle) kế thừa thực thể cơ sở Entity.

Giải pháp: Sử dụng tính chất kế thừa của Java để chia sẻ thuộc tính dùng chung và định nghĩa thuộc tính đặc thù riêng biệt.

Lý do: Loại bỏ trùng lặp mã nguồn (DRY), giúp cấu trúc nghiệp vụ rõ ràng, dễ bảo trì và dễ mở rộng khi thêm danh mục mặt hàng mới.

3.2 Áp dụng các tính chất OOP (1.0 điểm) - Bắt buộc

Hiện trạng: Cài đặt đầy đủ 4 tính chất OOP (Encapsulation, Inheritance, Polymorphism, Abstraction).

Giải pháp: Đóng gói (private fields, validate Regex nghiêm ngặt), Kế thừa (User/Item), Đa hình (Factory Method và ghi đè phương thức toString), Trừu tượng (abstract class User, Item).

Lý do: Đảm bảo tính toàn vẹn dữ liệu, dễ dàng mở rộng và tối ưu hóa khả năng tái sử dụng.

3.3 Design Patterns phù hợp (1.0 điểm) - Bắt buộc

Hiện trạng: Tích hợp thành công Factory Method và Observer Pattern vào luồng nghiệp vụ cốt lõi.

Giải pháp: ItemFactory và các lớp ArtFactory, VehicleFactory... sinh sản phẩm đa hình. Observer Pattern (cài đặt BidObserver qua ClientHandler) tự động gửi gói tin Socket khi có cập nhật giá mới.

Lý do: Giảm thiểu sự liên kết cứng (tight coupling) giữa các lớp, cung cấp giải pháp đồng bộ realtime tối ưu.

3.4 Quản lý người dùng, sản phẩm (1.0 điểm) - Bắt buộc

Hiện trạng: Hỗ trợ xác thực đăng nhập theo vai trò, đăng ký hồ sơ, nạp tiền ví; Seller CRUD sản phẩm; Admin kiểm duyệt phiên đấu giá.

Giải pháp: Phân quyền vai trò người dùng (Role ENUM) trong MySQL, thiết kế màn hình JavaFX ẩn hiện tính năng tương thích.

Lý do: Kiểm soát an ninh dữ liệu chặt chẽ và cá nhân hóa trải nghiệm người dùng cuối.

3.5 Chức năng đấu giá cốt lõi (1.0 điểm) - Bắt buộc

Hiện trạng: Điều hành vòng đời phiên đấu giá từ PENDING (đang chờ duyệt), RUNNING (đang diễn ra) đến FINISHED (đã kết thúc).

Giải pháp: BiddingService kiểm tra ví người dùng, bước giá tối thiểu (bidIncrement), và cấm Seller đặt giá trên sản phẩm sở hữu.

Lý do: Đảm bảo quy trình đấu giá diễn ra minh bạch, công bằng và chính xác tuyệt đối về số liệu tài chính.

3.6 Xử lý lỗi & Ngoại lệ tập trung (1.0 điểm) - Bắt buộc

Hiện trạng: Hệ thống không bị crash đột ngột khi gặp lỗi database hoặc dữ liệu đầu vào sai định dạng.

Giải pháp: Thiết lập ngoại lệ tùy chỉnh BusinessException để quản lý lỗi nghiệp vụ, try-catch-finally đóng kết nối an toàn, ghi nhật ký bằng SLF4J.

Lý do: Nâng cao độ bền bỉ, an toàn hệ thống, giúp dễ dàng debug và phát hiện lỗi vận hành.

3.7 Xử lý đấu giá đồng thời - Concurrency Control (1.0 điểm) - Bắt buộc

Hiện trạng: Tránh xung đột số dư tài khoản và giá cao nhất khi có hàng trăm lượt bid được gửi lên tại cùng một thời điểm.

Giải pháp: Áp dụng cơ chế Khóa bi quan (Pessimistic Locking) trong MySQL qua câu lệnh SELECT FOR UPDATE cho bản ghi auctions và users, kết hợp bọc trong một Transaction duy nhất.

Lý do: Đảm bảo tính nhất quán dữ liệu tuyệt đối (ACID) ở mức cơ sở dữ liệu, loại bỏ hoàn toàn lỗi Race Condition.

3.8 Cập nhật thời gian thực - Realtime Update (0.5 điểm) - Bắt buộc

Hiện trạng: Truyền giá đấu và người dẫn đầu tức thì tới các client đang theo dõi.

Giải pháp: Sử dụng TCP Socket Server kết hợp Executor Thread Pool (50 luồng) điều phối ClientHandler, truyền dữ liệu JSON qua Gson.

Lý do: Kết nối Socket nhẹ, tiết kiệm băng thông và đảm bảo độ trễ ở mức mili-giây.

3.9 Kiến trúc Client-Server (0.5 điểm) - Bắt buộc

Hiện trạng: Phân tách độc lập hoàn toàn giao diện người dùng và cơ sở dữ liệu.

Giải pháp: Client chỉ đảm nhận hiển thị và gửi thông điệp mạng, Server xử lý logic nghiệp vụ và lưu trữ dữ liệu.

Lý do: Nâng cao bảo mật cho backend, cho phép nâng cấp độc lập từng phân hệ.

3.10 Áp dụng mô hình MVC (0.5 điểm) - Bắt buộc

Hiện trạng: Cấu trúc mã nguồn JavaFX rõ ràng, khoa học.

Giải pháp: Phân rã dự án thành View (FXML/CSS), Controller (xử lý sự kiện UI), Model (Entity) và DAO (truy vấn cơ sở dữ liệu JDBC).

Lý do: Giúp code tách bạch, dễ viết unit test và thuận tiện phân công công việc.

3.11 Maven & Coding Convention (0.5 điểm) - Bắt buộc

Hiện trạng: Đồng bộ thư viện sử dụng và phong cách lập trình.

Giải pháp: Quản lý thư viện tập trung qua pom.xml, đặt tên biến chuẩn CamelCase và bảo mật biến môi trường qua file cấu hình .env.

Lý do: Đảm bảo dự án dễ dàng đóng gói, cài đặt đồng nhất ở mọi môi trường chạy máy tính.

3.12 Unit Test - JUnit 5 & Mockito (0.5 điểm) - Bắt buộc

Hiện trạng: Đảm bảo tính đúng đắn của logic nghiệp vụ tính.

Giải pháp: Viết kịch bản kiểm thử tự động với JUnit 5 kết hợp Mockito giả lập DAO, kiểm thử UserService, BiddingService.

Lý do: Phát hiện lỗi sớm khi sửa đổi code, bảo vệ tính ổn định của luồng xử lý chính.

3.13 CI/CD Pipeline - GitHub Actions (0.5 điểm) - Bắt buộc

Hiện trạng: Tự động hóa tích hợp và phát hành ứng dụng.

Giải pháp: Cấu hình ci.yml chạy test khi push/PR lên main, cd.yml tự động đóng gói Client thành ZIP và build Docker Server lên GHCR.

Lý do: Tiết kiệm tối đa thời gian triển khai sản phẩm, loại bỏ sai sót thao tác thủ công.

3.14 Đấu giá tự động (Auto-Bidding / Proxy Bidding) - Tùy chọn cộng 0.5 điểm

Hiện trạng: Tự động đặt giá hộ người dùng khi có đối thủ cạnh tranh trả giá cao hơn.

Giải pháp: Thuật toán resolveProxyBid() so sánh hạn mức maxLimit, tự đặt giá bằng giá cao thứ hai cộng bước giá (bidIncrement).

Lý do: Đây là chuẩn mực của sàn đấu giá lớn (như eBay), nâng cao trải nghiệm người dùng tối đa.

3.15 Chống bắn tĩa giá cuối giờ (Anti-Sniping) - Tùy chọn cộng 0.5 điểm

Hiện trạng: Ngăn chặn người dùng cố tình bid ở giây cuối cùng khiến đối thủ không kịp phản ứng.

Giải pháp: Kiểm tra nếu có lượt bid hợp lệ trong 5 phút cuối, phương thức applyAntiSniping() tự động dời thời gian kết thúc lùi thêm 30 phút.

Lý do: Đảm bảo quy trình cạnh tranh công bằng và tối ưu hóa doanh thu cho người bán.

3.16 Trực quan hóa lịch sử đấu giá (Bid History Visualization) - Tùy chọn cộng 0.5 điểm

Hiện trạng: Hiển thị tiến trình thay đổi giá cả trực quan cho người dùng dễ giám sát.

Giải pháp: Tích hợp LineChart/AreaChart của JavaFX Client, cập nhật tọa độ XYChart động từ gói tin Socket thời gian thực.

Lý do: Giao diện hiện đại, dễ theo dõi tiến trình tăng giá trực quan thay vì đọc bảng dữ liệu khô khan.

4. PHÂN CHIA CÔNG VIỆC TRONG NHÓM & KẾT LUẬN

Để hoàn thành khối lượng lớn dự án đúng tiến độ, Nhóm 4 đã phân rã công việc khoa học theo năng lực của từng thành viên:

Thành viên	Nhiệm vụ chính đảm nhiệm trong dự án	Đóng góp
Nguyễn Thành Lộc (Trưởng nhóm)	- Lập trình cổng REST API trên Java HttpServer sử dụng Gson. - Thiết lập hệ thống CI/CD tự động bằng GitHub Actions. - Đóng gói ứng dụng server qua Docker và Docker Compose. - Lập trình TCP Socket Server & Client đa luồng, cơ chế Observer.	100% (Tốt)
Nguyễn Phúc Bình	- Thiết kế giao diện người dùng JavaFX bằng FXML và CSS. - Tích hợp các Controller điều phối sự kiện giao diện Client. - Thiết kế các biểu đồ LineChart trực quan hóa lịch sử đặt giá (Bid History). - Tối ưu hóa trải nghiệm người dùng tương tác.	100% (Tốt)
Nguyễn Hải Anh	- Thiết kế kiến trúc hệ thống tổng thể. - Cài đặt xử lý đấu giá đồng thời (Concurrency Control và khóa bị quan). - Phát triển thuật toán đặt giá tự động (Auto-Bidding) và chống bắn tỉa (Anti-Sniping).	100% (Tốt)
Trần Minh Trung	- Xây dựng cơ sở dữ liệu MySQL. - Lập trình các lớp ánh xạ dữ liệu tầng DAO và Entity Model. - Phát triển Service xử lý logic nghiệp vụ tĩnh. - Viết kịch bản kiểm thử đơn vị Unit Test bằng JUnit 5 và Mockito.	100% (Tốt)

Trưởng nhóm

Nguyễn Thành Lộc